

Relatório do Projeto Prático: Simulação e Avaliação de Algoritmos de Escalonamento de CPU (**ossim**)

1. Identificação dos Autores

- **Nome do Estudante:** Tomás Rocha
- **Número de Aluno:** a22509631
- **Unidade Curricular:** Sistemas Operativos
- **Instituição:** Universidade Lusófona (Lisboa)

2. Repositório GitHub do Trabalho

O código-fonte completo do simulador, incluindo os módulos de escalonamento modificados (**scheduler.c**), as *workloads* de teste customizadas e os scripts de validação em Python, encontra-se alojado no seguinte repositório oficial:

- **Repositório GitHub:** <https://github.com/TomasfcRocha/simulador-escalonamento-so>

3. Tabelas de Desempenho e Tempos de Execução / Resposta por Cenário

A. Avaliação Direta FIFO vs. SJF (**demonstrar_sjf.py**)

Comparação detalhada do impacto da ordenação de processos por burst utilizando a carga padrão de teste:

Algoritmo	Tipo de Escalonamento	Tempo Médio de Espera	Turnaround Médio
FIFO	Não Preemptivo	9.40	13.60
SJF	Não Preemptivo	4.80	9.00

B. Avaliação Global de Todos os Algoritmos (**demonstrar_rr.py**)

Comparação abrangente incorporando algoritmos preemptivos e métricas de tempo de resposta:

Algoritmo	Tipo de Escalonamento	Tempo Médio de Espera	Turnaround Médio	Tempo Médio de Resposta
FIFO	Não Preemptivo	8.40	12.00	8.40
SJF	Não Preemptivo	8.40	12.00	8.40
Round Robin (RR)	Preemptivo (Quantum = 2)	4.40	8.00	2.80
MLFQ	Preemptivo Adaptativo	5.60	9.20	0.00

C. Avaliação de Cenários Específicos do MLFQ ([rodar_mlfq_cenarios.py](#))

Resultados obtidos com as *workloads* personalizadas para testar o comportamento dinâmico e o isolamento de processos interativos:

Cenário de Execução	Ficheiro de Carga	Tempo Médio de Espera	Turnaround Médio	Tempo Médio de Resposta
Cenário 1	cenario1.csv (Misto: Interativos + <i>Batch</i>)	9.40	13.60	2.00
Cenário 2	cenario2.csv (Comportamento Dinâmico / Alternado)	2.00	7.33	0.00

4. Análise e Comparação dos Resultados da Simulação

A. FIFO vs. SJF (O Impacto do Efeito Convoy)

- **Funcionamento:** O FIFO gere os processos estritamente por ordem de chegada, sofrendo visivelmente do **Efeito Convoy** (onde um processo pesado inicial bloqueia

as tarefas seguintes, elevando a espera média para 9.40). O **SJF**, ao seleccionar o processo com menor *burst* de CPU disponível, reduz drasticamente a espera média para 4.80, provando ser significativamente mais eficiente em termos globais quando os tempos de execução são conhecidos à partida.

B. O Salto de Desempenho com Round Robin (RR) e MLFQ

- **Funcionamento:** Através do uso de fatias de tempo (*quantum*) e preempção, estes algoritmos impedem que tarefas longas monopolizem o processador.
- **Análise de Resposta:** Os resultados demonstram que o **Round Robin** e o **MLFQ** alcançam tempos de resposta excepcionais (2.80 e 0.00, respetivamente), superando largamente o FIFO e o SJF e garantindo uma experiência interativa fluida sem bloqueios de sistema.

C. Dinamismo e Adaptação do MLFQ nos Cenários Práticos

- **Cenário 1 (cenario1.csv):** Demonstra o isolamento de tarefas. Os processos de curta duração terminam de forma célere, enquanto o processo *batch* esgota sucessivamente o seu *quantum* e é despromovido para as filas de menor prioridade.
- **Cenário 2 (cenario2.csv):** Valida a versatilidade do escalonador perante reentradas cíclicas de processos. Quando a aplicação exige computação intensiva, desce na hierarquia de filas; quando reduz a sua atividade, beneficia instantaneamente da reatividade das filas de topo.